

services\data\data_source_config.py

```
# Operating system interface for file and directory operations
import os
# Date and time operations for temporal boundary calculations
from datetime import datetime, timezone
# Type annotations for structured return values
from typing import Dict, Tuple
# Application configuration for directory paths and settings
import config

class DataSourceConfig:
    """Dynamic data source configuration based on current date"""

    def __init__(self):
        # Capture current UTC timestamp for consistent temporal calculations
        self.current_date = datetime.now(timezone.utc)
        # Initialize cache variables for performance optimization
        self._cutoff_cache = None
        self._cache_date = None

    def get_data_source_cutoff(self) -> Tuple[Dict, str]:
        """
        Calculate dynamic cutoff between historical and API data
        Returns: (cutoff_info, explanation)
        """
        # Cache the cutoff calculation for the current day to improve performance
        today = self.current_date.date()
        if self._cutoff_cache and self._cache_date == today:
            return self._cutoff_cache

        # Extract current year and month for boundary calculations
        current_year = self.current_date.year
        current_month = self.current_date.month

        # Previous month is the cutoff for historical data availability
        if current_month == 1:
            # January - previous month is December of last year
            historical_cutoff_year = current_year - 1
            historical_cutoff_month = 12
            api_start_year = current_year
            api_start_month = 1
        else:
            # Any other month - previous month of same year
            historical_cutoff_year = current_year
            historical_cutoff_month = current_month - 1
            api_start_year = current_year
            api_start_month = current_month

        # Build comprehensive cutoff information structure
        cutoff_info = {
            'historical': {
                'end_year': historical_cutoff_year,
                'end_month': historical_cutoff_month,
                'years_range': list(range(2010, historical_cutoff_year + 1))
            },
            'api': {
                'start_year': api_start_year,
                'start_month': api_start_month
            },
            'current': {
                'year': current_year,
```

```

'month': current_month,
'date': today
}
}

```

```

# Generate human-readable explanation for logging and debugging
explanation = (
    f"Historical data: 2010 ?
    {historical_cutoff_year}-{historical_cutoff_month:02d}, "
    f"API data: {api_start_year}-{api_start_month:02d} onwards"
)

```

```

# Cache the result for same-day reuse
self._cutoff_cache = (cutoff_info, explanation)
self._cache_date = today
return cutoff_info, explanation

```

```

def should_use_historical(self, year: int, month: int = None) -> bool:
    """

```

```

    Determine if given year/month should use historical data
    """

```

```

    cutoff_info, _ = self.get_data_source_cutoff()
    historical_end = cutoff_info['historical']

```

```

# Years clearly before cutoff use historical data
if year < historical_end['end_year']:
    return True
elif year == historical_end['end_year']:
    # For cutoff year, check month if provided
    if month is None:
        return True # Conservative approach for year-only queries
    return month <= historical_end['end_month']
else:
    # Years after cutoff use API data
    return False

```

```

def should_use_api(self, year: int, month: int = None) -> bool:
    """

```

```

    Determine if given year/month should use API data
    """

```

```

# Delegate to historical check and negate for consistency
return not self.should_use_historical(year, month)

```

```

def get_available_historical_files(self) -> Dict[int, str]:
    """

```

```

    Get list of available historical data files
    """

```

```

    available_files = {}

```

```

# Check if historical directory exists
if not os.path.exists(config.NVD_HISTORICAL_DIR):
    print(f"[DataSource] Historical directory does not exist:
    {config.NVD_HISTORICAL_DIR}")
    return available_files

```

```

# Check for different file patterns to accommodate various naming
conventions

```

```

patterns = [
    "CVE-{year}.json", # Simple naming
    "nvdcve-1.1-{year}.json", # JSON 1.1 format
    "nvdcve-2.0-{year}.json", # JSON 2.0 format
    "CVE-{year}.json.gz", # Compressed simple naming
    "nvdcve-1.1-{year}.json.gz", # Compressed JSON 1.1
    "nvdcve-2.0-{year}.json.gz" # Compressed JSON 2.0

```

```
]
```

```
# Check years from 2010 to current year for comprehensive coverage
current_year = self.current_date.year
for year in range(2010, current_year + 1):
    for pattern in patterns:
        filename = pattern.format(year=year)
        filepath = os.path.join(config.NVD_HISTORICAL_DIR, filename)
        if os.path.exists(filepath):
            available_files[year] = filepath
            break # Use first matching pattern for each year
```

```
return available_files
```

```
def log_data_source_status(self):
    """
```

```
Log current data source configuration for monitoring and debugging
    """
```

```
cutoff_info, explanation = self.get_data_source_cutoff()
available_files = self.get_available_historical_files()
```

```
# Log basic configuration information
print(f"[DataSource] Current date:
{self.current_date.strftime('%Y-%m-%d')}")
print(f"[DataSource] Configuration: {explanation}")
print(f"[DataSource] Historical directory: {config.NVD_HISTORICAL_DIR}")
print(f"[DataSource] Historical files available: {len(available_files)}
years")
```

```
# Log detailed file availability information
if available_files:
    print(f"[DataSource] Historical years: {min(available_files.keys())} -
{max(available_files.keys())}")
    print(f"[DataSource] Available files:")
    for year, filepath in sorted(available_files.items()):
        filename = os.path.basename(filepath)
        # Get file size for monitoring storage and transfer issues
        file_size = os.path.getsize(filepath) if os.path.exists(filepath) else 0
        print(f" {year}: {filename} ({file_size:,} bytes)")
    else:
        print(f"[DataSource] WARNING: No historical files found!")
```

```
# Check for missing files and alert about gaps in data coverage
expected_years = list(range(2010, cutoff_info['historical']['end_year'] +
1))
missing_years = [year for year in expected_years if year not in
available_files]
if missing_years:
    print(f"[DataSource] WARNING: Missing historical files for years:
{missing_years}")
```

```
# Global instance for application-wide data source configuration
data_source_config = DataSourceConfig()
```